

README

Exercise 1 Wekinator Project

Gesture-Based Regression with Wekinator
Processing Input, Wekinator Regression Model,
and Processing Output

Wekinator Project

Contents

1 Purpose of the Wekinator Project	3
2 Complete System Overview	3
3 Why This Is a Regression Problem	3
4 Wekinator Project Configuration	4
4.1 Receiving OSC Settings	4
4.2 Input Settings	4
4.3 Output Settings	5
5 Final Wekinator Setup Summary	5
6 Relationship with the Input Processing Sketch	5
7 Relationship with the Output Processing Sketch	6
8 Model Type and Algorithm Note	7
9 How to Train the Wekinator Project	7
10 Important Concept: Inputs vs Outputs	7
10.1 Inputs	7
10.2 Outputs	8
11 Correct Training Plan	8
12 Step-by-Step Training Procedure	9
12.1 Step 1 — Open the Input Sketch	9
12.2 Step 2 — Open the Output Sketch	9
12.3 Step 3 — Open or Create the Wekinator Project	9
12.4 Step 4 — Set the First Output Target	9
12.5 Step 5 — Record the First Example Group	9
12.6 Step 6 — Change Output Values and Record More Groups	10
12.7 Step 7 — Train the Model	10
12.8 Step 8 — Run the Model	10
13 The Mistake We Found During the Process	10

14	How the Mistake Was Fixed	11
15	Important Training Advice	11
15.1	Do Not Keep the Same Outputs for Every Recording	11
15.2	Move the Mouse While Recording	11
15.3	Record in Different Regions	12
15.4	Use Different Speeds	12
16	How to Check If the Wekinator Project Is Correct	12
16.1	Check 1 — Input Sketch	12
16.2	Check 2 — Wekinator Input	12
16.3	Check 3 — Wekinator Output	12
16.4	Check 4 — Output Processing Sketch	13
17	Troubleshooting	13
17.1	Problem: Wekinator Does Not Receive Input	13
17.2	Problem: Output Sketch Does Not Change	13
17.3	Problem: Circle Stays Fixed at 0.500 and 0.500	14
17.4	Problem: Circle Always Goes to the Same Size and Alpha	14
18	Final Project Workflow	14
19	Final Files Needed for Submission	14
20	Summary	15

1. Purpose of the Wekinator Project

This Wekinator project is the machine learning part of **Lab 3 — Exercise 1: Gesture-Based Regression with Wekinator**.

The goal of the exercise is to use mouse gesture data from a Processing input sketch and map it to continuous visual changes in a Processing output sketch.

The complete system connects three parts:

```
Processing Input Sketch
    ↓
Wekinator Regression Model
    ↓
Processing Output Sketch
```

The input Processing sketch captures the mouse gesture and sends three numerical features to Wekinator:

```
input-1 → normalized mouse x position
input-2 → normalized mouse y position
input-3 → normalized mouse movement speed
```

Wekinator learns a regression model that maps these three inputs to two continuous outputs:

```
outputs-1 → circle size
outputs-2 → outline alpha / transparency
```

The output Processing sketch receives these two values and visualizes them as a circle whose size and outline transparency change in real time.

2. Complete System Overview

The system works as follows:

```
Mouse movement in Processing input window
    ↓
Input sketch calculates x, y, and speed
    ↓
Input sketch normalizes all three values between 0.0 and 1.0
    ↓
Input sketch sends the values to Wekinator using OSC
    ↓
Wekinator receives the 3 inputs
    ↓
Wekinator predicts 2 continuous outputs
    ↓
Output sketch receives the outputs using OSC
    ↓
Output sketch maps the outputs to circle size and outline alpha
    ↓
The circle changes continuously on screen
```

The key idea is that the gesture is not classified into fixed categories. Instead, the gesture is treated as a continuous input signal. Wekinator learns how that signal should influence continuous visual parameters.

3. Why This Is a Regression Problem

This exercise uses **regression**, not classification.

In classification, the output would be a discrete class label such as:

```
Class 1
```

```
Class 2
Class 3
```

For example, a classifier might decide whether a gesture is:

```
slow gesture
fast gesture
left gesture
right gesture
```

However, in this exercise we do not want fixed labels. We want smooth visual control.

The output values are continuous numbers:

```
0.0
0.25
0.5
0.75
1.0
```

These values allow the circle to change gradually.

For this project:

```
Regression input  → x, y, speed
Regression output → circle size, outline transparency
```

This is why Wekinator must be configured with **continuous numeric outputs**.

4. Wekinator Project Configuration

The Wekinator project must be created with these settings.

4.1. Receiving OSC Settings

```
Wekinator listening for inputs and control on port: 6448
```

This port must match the port used in the input Processing sketch.

The input sketch sends OSC messages to:

```
127.0.0.1:6448
```

127.0.0.1 means the local computer.

4.2. Input Settings

```
OSC message: /wek/inputs
# inputs: 3
```

The number of inputs must be exactly **3**, because the input Processing sketch sends exactly three values:

```
1. xNorm
2. yNorm
3. speedNorm
```

If Wekinator is set to 5 inputs or another number, the project will not match the Processing input sketch correctly.

Correct input setup:

```
Input OSC message: /wek/inputs
Number of inputs: 3
```

4.3. Output Settings

```
OSC message: /wek/outputs
Host: localhost
Port: 12000
# outputs: 2
Type: All continuous
```

The number of outputs must be exactly **2**, because the output Processing sketch expects two values:

1. circle **size**
2. outline alpha

Correct output setup:

```
Output OSC message: /wek/outputs
Output host: localhost
Output port: 12000
Number of outputs: 2
Output type: All continuous
```

The output sketch listens on port 12000, so Wekinator must send its results to the same port.

5. Final Wekinator Setup Summary

The final Wekinator settings should be:

```
Input port: 6448
Input OSC message: /wek/inputs
Number of inputs: 3

Output OSC message: /wek/outputs
Output host: localhost
Output port: 12000
Number of outputs: 2
Output type: All continuous
```

The meaning of each input and output is:

```
input-1 → normalized x position
input-2 → normalized y position
input-3 → normalized movement speed

outputs-1 → circle size
outputs-2 → outline alpha / transparency
```

6. Relationship with the Input Processing Sketch

The input Processing sketch is responsible for acquiring mouse gesture data.

It calculates three values:

```
xNorm = mouseX / (float) width;
yNorm = mouseY / (float) height;
speedNorm = speed / maxSpeed;
```

These values are normalized between 0.0 and 1.0.

The input sketch then creates an OSC message:

```
OscMessage msg = new OscMessage("/wek/inputs");
```

and adds the three values in this order:

```
msg.add(xNorm);  
msg.add(yNorm);  
msg.add(speedNorm);
```

This order is very important.

Wekinator receives them as:

```
input-1 = xNorm  
input-2 = yNorm  
input-3 = speedNorm
```

So when the mouse moves horizontally, `input-1` changes.

When the mouse moves vertically, `input-2` changes.

When the mouse moves faster, `input-3` increases.

The input sketch sends these values continuously to Wekinator using:

```
OSC address: /wek/inputs  
Destination port: 6448
```

7. Relationship with the Output Processing Sketch

The output Processing sketch listens for Wekinator predictions.

It receives OSC messages from:

```
OSC address: /wek/outputs  
Listening port: 12000
```

The output sketch expects two values:

```
outputs-1 → circle size  
outputs-2 → outline alpha
```

Inside the output sketch, these values are stored as:

```
circleSizeOutput = theOscMessage.get(0).floatValue();  
outlineAlphaOutput = theOscMessage.get(1).floatValue();
```

Then the values are mapped to visual ranges:

```
circleSize = map(circleSizeOutput, 0, 1, 30, 350);  
outlineAlpha = map(outlineAlphaOutput, 0, 1, 0, 255);
```

This means:

```
outputs-1 close to 0.0 → small circle  
outputs-1 close to 1.0 → large circle  
  
outputs-2 close to 0.0 → transparent outline  
outputs-2 close to 1.0 → strong visible outline
```

The output sketch does not perform learning. It only visualizes the two values predicted by Wekinator.

8. Model Type and Algorithm Note

The lab specification asks for a **K-Nearest Neighbors regressor with $k = 3$** .

In Wekinator, the expected ideal model setup is:

```
Model type: KNN Regression
k: 3
```

However, in the Wekinator version used during this implementation, the model configuration window for continuous numeric outputs only showed these options:

```
Neural Network
Linear/Polynomial Regression
```

The KNN regression option was not available in the visible dropdown for this continuous-output project. For this reason, the project was kept as a continuous regression project using the available **Neural Network** regression model.

This is still compatible with the main regression behavior of the exercise because:

```
The inputs are continuous.
The outputs are continuous.
The system learns a mapping from gesture features to visual parameters.
```

If another Wekinator version provides KNN regression for continuous outputs, the correct choice for the lab specification would be:

```
KNN Regression, k = 3
```

If KNN is not available, the important practical requirement is to keep the project as a **regression project with 3 inputs and 2 continuous outputs**.

9. How to Train the Wekinator Project

Training Wekinator means recording examples of the relationship between:

```
mouse gesture input → desired visual output
```

During training, the Processing input sketch sends the input values automatically. The user does not manually type input values.

The user manually sets the desired output values in Wekinator before recording.

For example:

```
outputs-1 = 0.2
outputs-2 = 0.2
```

means:

```
For this gesture, I want a small circle with a transparent outline.
```

Then Wekinator records the current input values together with the chosen output values.

10. Important Concept: Inputs vs Outputs

10.1. Inputs

Inputs come from the Processing input sketch.

They are controlled by moving the mouse.

The user does not type them manually.

```
input-1 = mouse x position
input-2 = mouse y position
input-3 = mouse speed
```

10.2. Outputs

Outputs are the target values that the user sets in Wekinator during training.

They represent what the user wants the visual result to be.

```
outputs-1 = desired circle size
outputs-2 = desired outline transparency
```

So the training process is:

```
Move the mouse to create input values.
Set output values to define the desired result.
Record the example.
```

Wekinator learns from these examples.

11. Correct Training Plan

A good training set should include different mouse regions and different output values.

The following training plan was used as the corrected approach:

```
Recording 1:
Mouse area: top-left
outputs-1 = 0.2
outputs-2 = 0.2
Meaning: small circle, transparent outline

Recording 2:
Mouse area: top-right
outputs-1 = 0.8
outputs-2 = 0.2
Meaning: large circle, transparent outline

Recording 3:
Mouse area: bottom-left
outputs-1 = 0.2
outputs-2 = 0.8
Meaning: small circle, strong outline

Recording 4:
Mouse area: bottom-right
outputs-1 = 0.8
outputs-2 = 0.8
Meaning: large circle, strong outline

Optional Recording 5:
Mouse area: center
outputs-1 = 0.5
outputs-2 = 0.5
Meaning: medium circle, medium outline
```

This creates a useful training structure because Wekinator sees different input gestures connected to different output targets.

12. Step-by-Step Training Procedure

12.1. Step 1 — Open the Input Sketch

Open and run the Processing input sketch:

```
Exercise1_GestureInput
```

Check that the screen shows:

```
x normalized
y normalized
speed normalized
```

Move the mouse inside the input sketch window. The values should change.

12.2. Step 2 — Open the Output Sketch

Open and run the Processing output sketch:

```
Exercise1_CircleOutput
```

The output sketch should show a black window with a circle in the center.

Before Wekinator is trained and running, the values may stay at:

```
Output 1 = 0.500
Output 2 = 0.500
```

This is normal because the sketch uses default values until it receives OSC messages from Wekinator.

12.3. Step 3 — Open or Create the Wekinator Project

Create the Wekinator project using:

```
3 inputs
2 outputs
/wek/inputs
/wek/outputs
input port 6448
output port 12000
```

Make sure the OSC In indicator becomes active when the input Processing sketch is running.

12.4. Step 4 — Set the First Output Target

For the first recording, set:

```
outputs-1 = 0.2
outputs-2 = 0.2
```

This means:

```
small circle
transparent outline
```

12.5. Step 5 — Record the First Example Group

Click:

```
Start Recording
```

Then move the mouse inside the input sketch window.

For the first group, move the mouse in the top-left area.

Move continuously for about 3–5 seconds.

Then click:

Stop Recording

The number of examples should increase.

12.6. Step 6 — Change Output Values and Record More Groups

Before each new recording, change the output target values.

For example:

```
Top-right area:
outputs-1 = 0.8
outputs-2 = 0.2

Bottom-left area:
outputs-1 = 0.2
outputs-2 = 0.8

Bottom-right area:
outputs-1 = 0.8
outputs-2 = 0.8

Center area:
outputs-1 = 0.5
outputs-2 = 0.5
```

For each group:

```
Set the output values.
Click Start Recording.
Move the mouse in the selected area.
Stop Recording.
```

12.7. Step 7 — Train the Model

After recording all example groups, click:

Train

Wekinator uses the recorded examples to learn the mapping between gesture inputs and visual outputs.

12.8. Step 8 — Run the Model

After training, click:

Run

Now Wekinator will automatically predict the two output values from the live mouse input.

At this stage, do not manually change the output values anymore.

Move the mouse inside the input sketch window and observe the output sketch.

The circle should change size and transparency continuously.

13. The Mistake We Found During the Process

During the first training attempt, the Wekinator settings were mostly correct, but the training data had one major problem.

The same output target values were used for all recordings:

```
outputs-1 = 0.2  
outputs-2 = 0.2
```

This meant that Wekinator learned:

```
Every mouse gesture → 0.2, 0.2
```

As a result, after training and running the model, the output values did not change. The circle stayed almost fixed.

This was not a bug in the Processing input sketch.

It was not a bug in the Processing output sketch.

It was a training-data problem.

The model cannot learn different visual responses if all training examples have the same target output.

14. How the Mistake Was Fixed

The fix was to record different mouse gestures with different output target values.

Instead of using:

```
0.2, 0.2  
0.2, 0.2  
0.2, 0.2  
0.2, 0.2
```

we used different target values such as:

```
0.2, 0.2  
0.8, 0.2  
0.2, 0.8  
0.8, 0.8  
0.5, 0.5
```

This gave Wekinator meaningful examples.

Now the model can learn that different mouse gestures should produce different visual outputs.

15. Important Training Advice

15.1. Do Not Keep the Same Outputs for Every Recording

If every training example has the same output values, the model will produce almost the same output all the time.

Correct approach:

```
Different gesture → different desired output
```

Wrong approach:

```
Different gesture → same desired output every time
```

15.2. Move the Mouse While Recording

The third input is mouse speed.

If the mouse is not moving, speed becomes close to:

```
0.0
```

Some zero-speed examples are normal, but too many still-mouse examples can make the speed feature less useful.

During each recording:

```
Click Start Recording.  
Immediately move the mouse.  
Keep moving for 3-5 seconds.  
Stop Recording.
```

15.3. Record in Different Regions

Because the model uses x and y , it is useful to record in different regions of the input sketch:

```
top-left  
top-right  
bottom-left  
bottom-right  
center
```

This helps Wekinator learn how position affects the outputs.

15.4. Use Different Speeds

Because the model also uses speed, it is useful to record different movement speeds:

```
slow movement  
medium movement  
fast movement
```

This helps Wekinator learn how speed affects the visual result.

16. How to Check If the Wekinator Project Is Correct

After training and clicking **Run**, test the system.

16.1. Check 1 — Input Sketch

Move the mouse inside the input Processing window.

The input sketch should show changing values:

```
x normalized  
y normalized  
speed normalized
```

If these values change, the input sketch is working.

16.2. Check 2 — Wekinator Input

In Wekinator, the **OSC In** indicator should show that it is receiving data.

When the mouse moves, the input values in Wekinator should update.

16.3. Check 3 — Wekinator Output

After clicking **Run**, Wekinator should generate output values automatically.

The outputs should not stay fixed forever.

Values may look like:

```
outputs-1 = 0.34  
outputs-2 = 0.71
```

or:

```
outputs-1 = 0.82
outputs-2 = 0.25
```

They do not need to be exactly the same as the training values. Regression outputs can be intermediate values.

16.4. Check 4 — Output Processing Sketch

The output sketch should show changing raw values:

```
Output 1 - circle size raw
Output 2 - outline alpha raw
```

The circle should also visually change:

```
Output 1 changes → circle size changes
Output 2 changes → outline transparency changes
```

The Processing console may also show messages such as:

```
Received from Wekinator -> size: 0.64 alpha: 0.82
```

This confirms that the output sketch is receiving OSC messages from Wekinator.

17. Troubleshooting

17.1. Problem: Wekinator Does Not Receive Input

Possible causes:

```
Input sketch is not running.
Wekinator input port is not 6448.
Input OSC message is not /wek/inputs.
The input sketch is sending to a different port.
```

Fix:

```
Run the input sketch.
Set Wekinator input port to 6448.
Set Wekinator input message to /wek/inputs.
```

17.2. Problem: Output Sketch Does Not Change

Possible causes:

```
Wekinator has not been trained.
Wekinator is not in Run mode.
Wekinator is sending to the wrong port.
Output sketch is not running.
All training examples have the same output values.
```

Fix:

```
Train the model.
Click Run.
Make sure Wekinator sends to port 12000.
Make sure the output message is /wek/outputs.
Record examples with different output target values.
```

17.3. Problem: Circle Stays Fixed at 0.500 and 0.500

This usually means the output sketch has not received any Wekinator output messages yet.

Fix:

```
Run Wekinator.
Click Train.
Click Run.
Make sure Wekinator output port is 12000.
Make sure output sketch is running.
```

17.4. Problem: Circle Always Goes to the Same Size and Alpha

This usually means all training examples had the same output targets.

Fix:

```
Delete recordings or create a new project.
Record again with different outputs for different gestures.
Train again.
Run again.
```

18. Final Project Workflow

The correct order for running the project is:

1. Open the Processing input sketch.
2. Run the Processing input sketch.
3. Open the Processing output sketch.
4. Run the Processing output sketch.
5. Open the Wekinator project.
6. Check that Wekinator receives OSC input.
7. Record examples with different output values.
8. Click Train.
9. Click Run.
10. Move the mouse in the input sketch window.
11. Observe the circle changing in the output sketch.
12. Save the Wekinator project.

19. Final Files Needed for Submission

For Exercise 1, the final submission should include:

1. Processing input project
2. Processing output project
3. Wekinator project
4. Short report / README discussion

Recommended folder structure:

```
Lab3_Exercise1/
  Exercise1_GestureInput/
    Exercise1_GestureInput.pde

  Exercise1_CircleOutput/
    Exercise1_CircleOutput.pde

  Wekinator_Project/
    Wekinator project files

README.md
```

20. Summary

This Wekinator project is the central machine learning component of Exercise 1.

It receives three real-time mouse gesture features from the Processing input sketch:

```
x position  
y position  
movement speed
```

It predicts two continuous visual parameters:

```
circle size  
outline alpha / transparency
```

The main technical settings are:

```
Inputs: 3  
Outputs: 2  
Input OSC message: /wek/inputs  
Input port: 6448  
Output OSC message: /wek/outputs  
Output port: 12000  
Output type: continuous regression
```

The most important lesson from the process was that the output values must be different for different training examples. If all examples use the same outputs, Wekinator will learn a constant result and the circle will not change.

After correcting the training data by assigning different target values to different mouse regions and movements, the model becomes able to produce changing regression outputs. These outputs control the size and transparency of the circle in the final Processing visualization.